



Stored Procedures

O que são Stored Procedures?

Definição:

- Uma **Stored Procedure** é um conjunto de comandos SQL armazenados no servidor, que pode ser executado repetidamente com parâmetros opcionais.
- Ela permite encapsular a lógica em um procedimento reutilizável.

Vantagens:

- Redução de tráfego entre cliente e servidor.
- Reutilização de código.
- Maior segurança e controle sobre operações.
- Otimização de performance.

Slide 3: Sintaxe Básica de uma Stored Procedure

Exemplo simples de sintaxe:

```
DELIMITER //
```

```
CREATE PROCEDURE nome_procedure()  
BEGIN  
    -- Corpo da procedure  
    SELECT 'Hello, Mundo!';  
END //
```

```
DELIMITER ;
```

Explicação:

- `DELIMITER` muda o delimitador padrão para permitir múltiplos comandos na procedure.
- `CREATE PROCEDURE` cria a procedure com o nome especificado.
- `BEGIN...END` marca o início e fim da procedure.
- Após a criação, você pode chamar a procedure usando `CALL nome_procedure();`.

Procedures com Parâmetros de Entrada

Parâmetros de entrada permitem passar valores para a procedure.

Exemplo:

```
DELIMITER //
```

```
CREATE PROCEDURE buscar_produto(IN nome_produto VARCHAR(100))
```

```
BEGIN
```

```
    SELECT * FROM produtos WHERE nome = nome_produto;
```

```
END //
```

```
DELIMITER ;
```

Explicação:

- **IN** indica que **nome_produto** é um parâmetro de entrada.
- O valor passado será usado na query **SELECT**.

Execução:

```
CALL buscar_produto('Produto A');
```

Procedures com Parâmetros de Saída

Parâmetros de saída permitem retornar valores para a aplicação.

Exemplo:

```
DELIMITER //
```

```
CREATE PROCEDURE calcular_preco_medio(IN categoria VARCHAR(50), OUT preco_medio DECIMAL(10, 2))
```

```
BEGIN
```

```
    SELECT AVG(preco) INTO preco_medio FROM produtos WHERE categoria = categoria;
```

```
END //
```

```
DELIMITER ;
```

Explicação:

- `OUT` especifica o parâmetro de saída `preco_medio`.
- O valor médio dos preços será armazenado em `preco_medio` e retornado.

Execução:

```
SET @resultado = 0;  
CALL calcular_preco_medio('Categoria 1', @resultado);  
SELECT @resultado;
```

Uso de Variáveis Locais

Variáveis podem ser usadas para armazenar valores temporários.

Sintaxe:

```
DECLARE nome_variavel tipo_variavel [DEFAULT valor_inicial];
```

Exemplo:

```
DELIMITER //
```

```
CREATE PROCEDURE atualizar_preco(IN categoria VARCHAR(50))  
BEGIN  
    DECLARE preco_novo DECIMAL(10, 2);  
  
    SELECT AVG(preco) INTO preco_novo FROM produtos WHERE categoria = categoria;  
  
    UPDATE produtos SET preco = preco_novo WHERE categoria = categoria;  
END //
```

```
DELIMITER ;
```

Estruturas Condicionais (IF, CASE)

Exemplo de uso do IF:

```
DELIMITER //
```

```
CREATE PROCEDURE verificar_estoque(IN produto_id INT, OUT resultado VARCHAR(50))  
BEGIN  
    DECLARE quantidade INT;  
    SELECT estoque INTO quantidade FROM produtos WHERE id = produto_id;  
    IF quantidade > 0 THEN  
        SET resultado = 'Em estoque';  
    ELSE  
        SET resultado = 'Fora de estoque';  
    END IF;  
END //
```

```
DELIMITER ;
```

Explicação:

- **IF** verifica se a condição é verdadeira e executa o bloco correspondente.

Exemplo de uso do CASE:

```
CASE
  WHEN valor1 THEN resultado1
  WHEN valor2 THEN resultado2
  ELSE resultado_default
END
```

Loops em Stored Procedures

```
DELIMITER //
CREATE PROCEDURE incrementar_estoque(IN categoria_param VARCHAR(50))
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE produto_id INT;
    DECLARE cur CURSOR FOR SELECT id FROM produtos WHERE categoria = categoria_param;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    OPEN cur;
loop_label: LOOP
    FETCH cur INTO produto_id;
    IF done THEN
        LEAVE loop_label;
    END IF;
    -- Aumentar o estoque em 10 para cada produto
    UPDATE produtos SET estoque = estoque + 10 WHERE id = produto_id;
END LOOP;
CLOSE cur;
END //

DELIMITER ;
```

Outras opções de loop:

- **WHILE loop:**

```
WHILE condição DO  
    -- Código  
END WHILE;
```

- **REPEAT loop:**

```
REPEAT  
    -- Código  
UNTIL condição END REPEAT;
```

Tratamento de Erro (HANDLERS)

Handlers permitem capturar erros ou condições específicas.

Exemplo:

```
DELIMITER //
```



```
CREATE PROCEDURE safe_atualizar_preco(IN categoria_param VARCHAR(50))  
BEGIN  
    DECLARE erro_ocorrido INT DEFAULT 0;  
  
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION  
        SET erro_ocorrido = 1;  
  
    -- Tentar executar a operação  
    UPDATE produtos SET preco = preco * 1.10 WHERE categoria = categoria_param;  
  
    IF erro_ocorrido THEN  
        SELECT 'Erro ao atualizar os preços.';  
    ELSE  
        SELECT 'Preços atualizados com sucesso.';  
    END IF;  
END //
```



```
DELIMITER ;
```

Explicação:

- `SQLException` captura qualquer erro SQL.
- O tratamento de erros garante que a aplicação ou sistema seja notificado caso algo dê errado.

Resumo e Considerações Finais

- **Stored Procedures** ajudam a encapsular lógica de negócio no banco de dados.
- Parâmetros de entrada/saída permitem maior flexibilidade.
- O uso de variáveis, condicionais e loops oferece controle sobre a execução.
- É importante implementar **tratamento de erro** para lidar com falhas.

Dica: Sempre testar as procedures em um ambiente de desenvolvimento antes de usar em produção.

Exercício 1: Índice Simples

Descrição: Crie uma tabela chamada `estudantes` com colunas `id`, `nome` e `email`. Em seguida, crie um índice simples na coluna `email`.

Exemplo:

```
CREATE TABLE estudantes (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100)  
);  
CREATE INDEX idx_email ON estudantes(email);
```

Exercício 2: Índice Composto

Descrição: Crie uma tabela chamada `notas` com colunas `id`, `estudante_id`, `disciplina` e `nota`. Crie um índice composto nas colunas `estudante_id` e `disciplina`.

Exemplo:

```
CREATE TABLE notas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    estudante_id INT,  
    disciplina VARCHAR(50),  
    nota DECIMAL(3, 2)  
);  
CREATE INDEX idx_estudante_disciplina ON notas(estudante_id, disciplina);
```

Exercício 3: Índice Único

Descrição: Crie uma tabela chamada `cursos` com colunas `id`, `nome_curso` e `codigo_curso`. Crie um índice único na coluna `codigo_curso`.

Exemplo:

```
CREATE TABLE cursos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome_curso VARCHAR(100),  
    codigo_curso VARCHAR(10) UNIQUE  
);  
CREATE UNIQUE INDEX idx_codigo_curso ON cursos(codigo_curso);
```

Exercício: Comparação de Consultas com e Sem Índice

Descrição: Crie uma tabela chamada `produtos` com colunas `id`, `nome`, `categoria` e `preco`. Insira um número significativo de registros (por exemplo, 10.000). Depois, faça duas consultas: uma sem índice na coluna `categoria` e outra com índice.

Passo a Passo:

1. Criar a tabela e inserir dados:

```
CREATE TABLE produtos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  categoria VARCHAR(50),  
  preco DECIMAL(10, 2)  
);
```

```

-- Inserir 10.000 produtos (exemplo simplificado)
INSERT INTO produtos (nome, categoria, preco)
SELECT
    CONCAT('Produto ', n) AS nome,
    CONCAT('Categoria ', (n % 10)) AS categoria,
    ROUND(RAND() * 100, 2) AS preco
FROM
    (SELECT @rownum := @rownum + 1 AS n
    FROM (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5) t1,
        (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5) t2,
        (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5) t3,
        (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5) t4,
        (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5) t5,
        (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5) t6,
        (SELECT @rownum := 0) r) num;

```

2. Consulta sem índice:

```
SELECT * FROM produtos WHERE categoria = 'Categoria 1';
```

3. Criar índice na coluna **categoria**:

```
CREATE INDEX idx_categoria ON produtos(categoria);
```

4. Consulta com índice:

```
SELECT * FROM produtos WHERE categoria = 'Categoria 1';
```

Objetivo: Meça o tempo de execução das duas consultas. Isso demonstrará a diferença de desempenho ao usar índices.

Exercício 1: Procedure com Parâmetro de Entrada

Descrição: Crie uma stored procedure chamada `buscar_produto_por_categoria` que receba uma categoria como parâmetro e retorne todos os produtos dessa categoria.

Exemplo:

```
DELIMITER //
```

```
CREATE PROCEDURE buscar_produto_por_categoria(IN categoria_param VARCHAR(50))  
BEGIN  
    SELECT * FROM produtos WHERE categoria = categoria_param;  
END //
```

```
DELIMITER ;
```

Tarefa: Execute a procedure `buscar_produto_por_categoria` para a categoria "Categoria 1".

```
CALL buscar_produto_por_categoria('Categoria 1');
```

Exercício 2: Procedure com Parâmetro de Entrada e Saída

Descrição: Crie uma stored procedure chamada `calcular_preco_medio_categoria` que receba o nome de uma categoria como entrada e retorne o preço médio dos produtos dessa categoria como saída.

Exemplo:

```
DELIMITER //
```

```
CREATE PROCEDURE calcular_preco_medio_categoria(  
    IN categoria_param VARCHAR(50),  
    OUT preco_medio DECIMAL(10, 2)  
)  
BEGIN  
    SELECT AVG(preco) INTO preco_medio  
    FROM produtos  
    WHERE categoria = categoria_param;  
END //
```

```
DELIMITER ;
```

Tarefa: Calcule o preço médio dos produtos da categoria "Categoria 1" e armazene o resultado em uma variável.

```
SET @preco_medio = 0;  
CALL calcular_preco_medio_categoria('Categoria 1', @preco_medio);  
SELECT @preco_medio;
```

Exercício 3: Procedure com Loop

Descrição: Crie uma stored procedure chamada `atualizar_preco_por_categoria` que, para uma determinada categoria, aumente o preço de todos os produtos em 10%. Utilize um loop para processar os produtos dessa categoria.

Exemplo:

```
DELIMITER //
```

```
CREATE PROCEDURE atualizar_preco_por_categoria(IN categoria_param VARCHAR(50))
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE produto_id INT;
    DECLARE preco_atual DECIMAL(10, 2);
    DECLARE cur CURSOR FOR
        SELECT id, preco FROM produtos WHERE categoria = categoria_param;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO produto_id, preco_atual;
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Atualizar o preço em 10%
        UPDATE produtos SET preco = preco_atual * 1.10 WHERE id = produto_id;
    END LOOP;

    CLOSE cur;
END //
```

```
DELIMITER ;
```

Tarefa: Execute a procedure `atualizar_preco_por_categoria` para a categoria "Categoria 1" e observe o aumento de 10% nos preços dessa categoria.

```
CALL atualizar_preco_por_categoria('Categoria 1');
```